

【例 3-1】 下面是一个有连接的编程实例。程序分两部分：服务器端程序和客户端程序。

(1) 服务器端程序。

```
//TCPdtd_server.cpp -main, TCPdaytimed
#include<stdlib.h>
#include<stdio.h>
#include<winsock2.h>
#include<time.h>

void errexit(const char *,...);
void TCPdaytimed(SOCKET);
SOCKET passiveTCP(const char *, int);

#define QLEN    5
#define WSVERS  MAKEWORD(2, 0)

/* -----
 * main - Iterative TCP server for DAYTIME service
 * -----
 */
void
main(int argc, char * argv[])
/* argc: 命令行参数个数, 例如:C:\>TCPdaytimed 8080,  argc=2 argv[0]="TCPdaytimed",
argv[1]="8080" */
{
    struct    sockaddr_in fsin;          /* the from address of a client */
    char * service="daytime";           /* service name or port number */
    SOCKET msock, ssock;                /* master & slave sockets */
    int alen;                           /* from-address length */
    WSADATA wsadata;                    //Socket 的版本信息

    switch(argc) {
    case 1:                             //命令行仅文件名 (即命令动词)
        break;
    case 2:                             //命令行仅一个参数
        service=argv[1];
        break;
    default:                             //其他情况
        errexit("usage: TCPdaytimed [port]\n");
    }

    if (WSAStartup(WSVERS, &wsadata) !=0)
//Winsock 服务初始化。首参数指明程序请求使用的 Socket 版本 (高位字节指明副版本, 低位字
//节指明主版本); 次参数返回请求的 Socket 的版本信息
```

```

    errexit("WSAStartup failed\n");

    msock=passiveTCP(service, QLEN); //创建被动的套接字,调用 passiveTCP 实现。第
//一个是字符串:服务的名字或者端口号;第二个是传入连接的请求队列所需的长度
    while (1) {
        alen=sizeof(struct sockaddr);
        ssock=accept(msock, (struct sockaddr *)&fsin, &alen);
        if (ssock==INVALID_SOCKET)
            errexit("accept failed: error number %d\n",
                GetLastError());
        TCPdaytimed(ssock);
        (void) closesocket(ssock); //调用 close 是从容关闭:TCP 保证所有的数据可靠交
//付给客户(连接终止前收到确认)
    }
//在循环中,使用 accept 从主套接字得到一个连接(accept 完成三次握手过程)
//对于新的连接服务器调用过程 TCPdaytimed 进行处理
//处理完毕继续循环,再次调用 accept 阻塞
//循环实现就足够了,服务器忙时,其他的请求可以排队
} //main()

```

```

/* -----
 * TCPdaytimed -do TCP DAYTIME protocol
 * -----
 */
//从其他机器上获得另一个系统当前的日期和时间
Void TCPdaytimed(SOCKET fd)
{
    char *pts; // pointer to time string */
    time_t now; // current time */

    (void) time(&now);
    pts=ctime(&now);
    (void) send(fd, pts, strlen(pts), 0);
    printf("%s", pts);
}

```

(2) 客户端程序。

```

/* TCPdtc_client.cpp -main, TCPdaytime */

#include<stdlib.h>
#include<stdio.h>
#include<winsock2.h>

void TCPdaytime(const char *, const char *);
void errexit(const char *, ...);

```

```
SOCKET connectTCP(const char *, const char *);
```

```
#define LINELEN      128
```

```
#define WSVERS      MAKEWORD(2, 0)
```

```
/* -----
```

```
* main -TCP client for DAYTIME service
```

```
* -----
```

```
*/
```

```
int
```

```
main(int argc, char * argv[])
```

```
/* usage: TCPdaytime [host [port]]\n */
```

```
{ /* host 可以为服务器的 IP 地址或域名, port 可  
    以为服务器监听的端口号或服务名 */
```

```
    char * host="localhost";
```

```
/* host to use if none supplied */
```

```
    char * service="daytime";
```

```
/* default service port */
```

```
    WSADATA wsadata;
```

```
    switch(argc) {
```

```
    case 1:
```

```
        host="localhost";
```

```
        break;
```

```
    case 3:
```

```
        service=argv[2];
```

```
/* FALL THROUGH */
```

```
    case 2:
```

```
        host=argv[1];
```

```
        break;
```

```
    default:
```

```
        fprintf(stderr, "usage: TCPdaytime [host [port]]\n");
```

```
        exit(1);
```

```
    }
```

```
    if (WSAStartup(WSVERS, &wsadata)!=0)
```

```
/* 启动某版本的 DLL */
```

```
        errexit("WSAStartup failed\n");
```

```
    TCPdaytime(host, service);
```

```
/* 建立连接并传输数据 */
```

```
    WSACleanup();
```

```
/* 卸载某版本的 DLL */
```

```
    printf("按任意键继续...");
```

```
    getchar();
```

```
    return 0;
```

```
/* exit */
```

```
}
```

```
/* -----
```

```
* TCPdaytime -invoke Daytime on specified host and print results
```

```
* -----
```

```
*/
```

```
void
```

```

TCPdaytime(const char * host, const char * service)
{
    char buf[LINELEN+1];                /* buffer for one line of text */
    SOCKET s;                          /* socket descriptor */
    int cc;                             /* recv character count */

    s=connectTCP(host, service);        /* 建立连接 */

    cc=recv(s, buf, LINELEN, 0);
    while(cc!=SOCKET_ERROR && cc > 0) {
        buf[cc]='\0';                    /* ensure null-termination */
        (void) fputs(buf, stdout);
        cc=recv(s, buf, LINELEN, 0);
    }
    closesocket(s);
}

```

(3) 程序运行。

为了运行程序,须先编译程序:在 Visual C++ 下建立 dsw 文件,然后生成并运行(按 Ctrl+F5 键)。使用套接字函数的程序链接时需要加载 ws2_2.lib,即在工程设置/Link/General/对象/库模块中加入 ws2_2.lib。

运行程序时要先运行服务器程序:在命令窗口中输入 TCPdte_server server_ip [server_port]\n;在另一台计算机上运行 TCPdte_client server_ip [server_port]\n。如果程序没有回应,可能是防火墙引起服务器的监听端口关闭。在程序运行时可以启动 Wireshark 捕获数据包,通过分析可以展示 C/S 的连接过程。

【例 3-2】 将网卡的工作模式设置为混杂模式。网卡的混杂模式是网络分析的需要。Linux 已经提供了设置网卡为混杂模式的命令,实际上也可自行编写程序实现。虽然只是一个本地应用,因涉及网络接口,也要使用 socket 编程。

(1) 程序清单。

```

/* -----
 * 程序:makprom
 * 功能:设置网卡为混杂模式
 * -----
 */
#include<stdio.h>
#include<sys/ioctl.h>
#include<stdlib.h>
#include<sys/socket.h>
#include<cstring.h>
#include<linux/in.h>
#include<linux/if_ether.h>
#include<unistd.h>
#include<net/if.h>

```